

INGENIERÍA DE SOFTWARE 1 · 2016701

biUNestar

Análisis técnico del proyecto: arquitectura, modelo de datos y cumplimiento de requisitos

EQUIPO

Daniel Felipe Agudelo Parra

Jaime Javier Andrés Quiñones

Omar Andrés Quiñones

Pedro Alejandro Castiblanco Castañeda

¿Qué es biUNestar?

Una aplicación para que estudiantes de la Universidad Nacional registren y hagan seguimiento a su bienestar: **sueño, hidratación, actividad física y estado de ánimo.**

Objetivo

Promover el autocuidado y fortalecer hábitos saludables mediante reportes y recordatorios.

Usuarios del sistema

Estudiante registra hábitos, ve su progreso y genera reportes

Administrador gestiona recursos de apoyo y perfiles de estudiantes

El proyecto vive en 3 capas

Presentación

Lógica de negocio

Persistencia de datos

Arquitectura general — patrón MVT

Presentación

`templates/` (Django Templates + Bootstrap 5)

Renderiza HTML en el servidor; crispy-forms para formularios

Lógica de negocio

`views.py` + `forms.py` por app

Procesa peticiones, valida datos, coordina modelos y templates

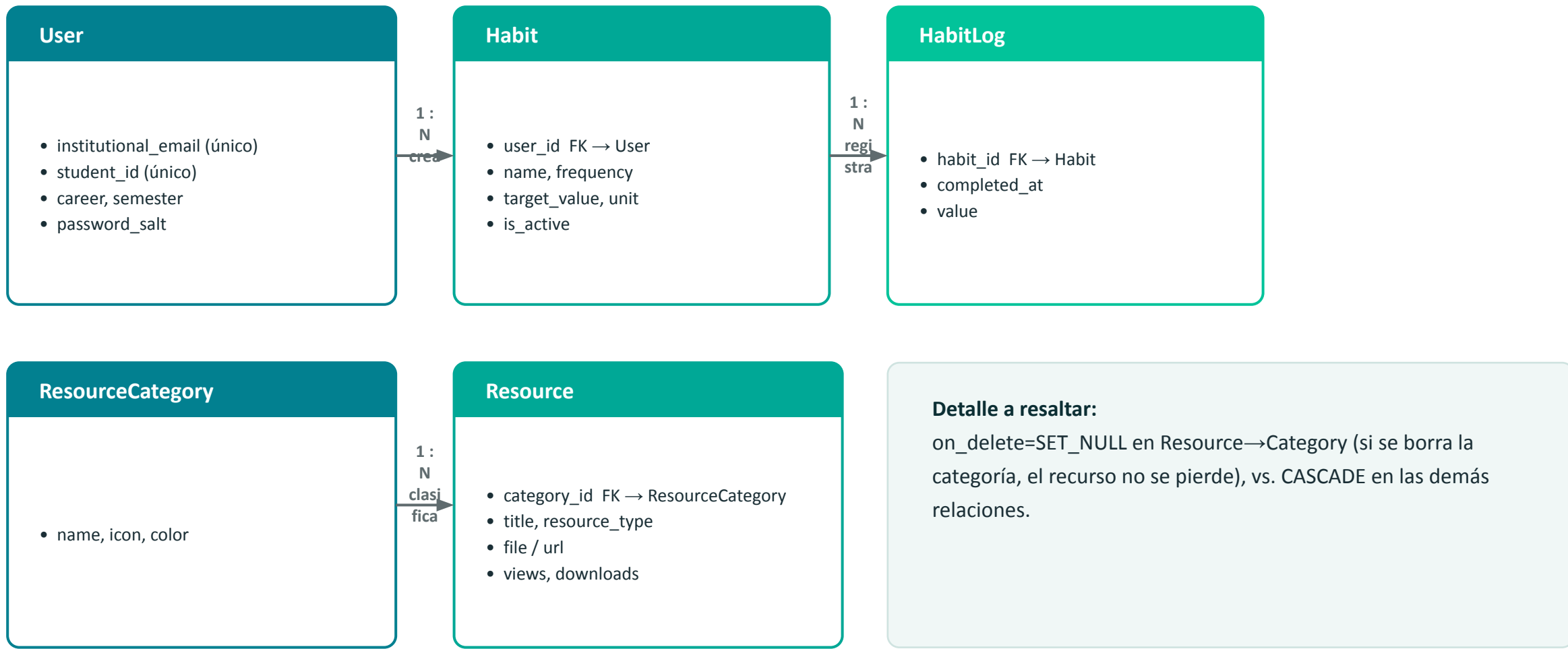
Persistencia de datos

`models.py` + ORM de Django

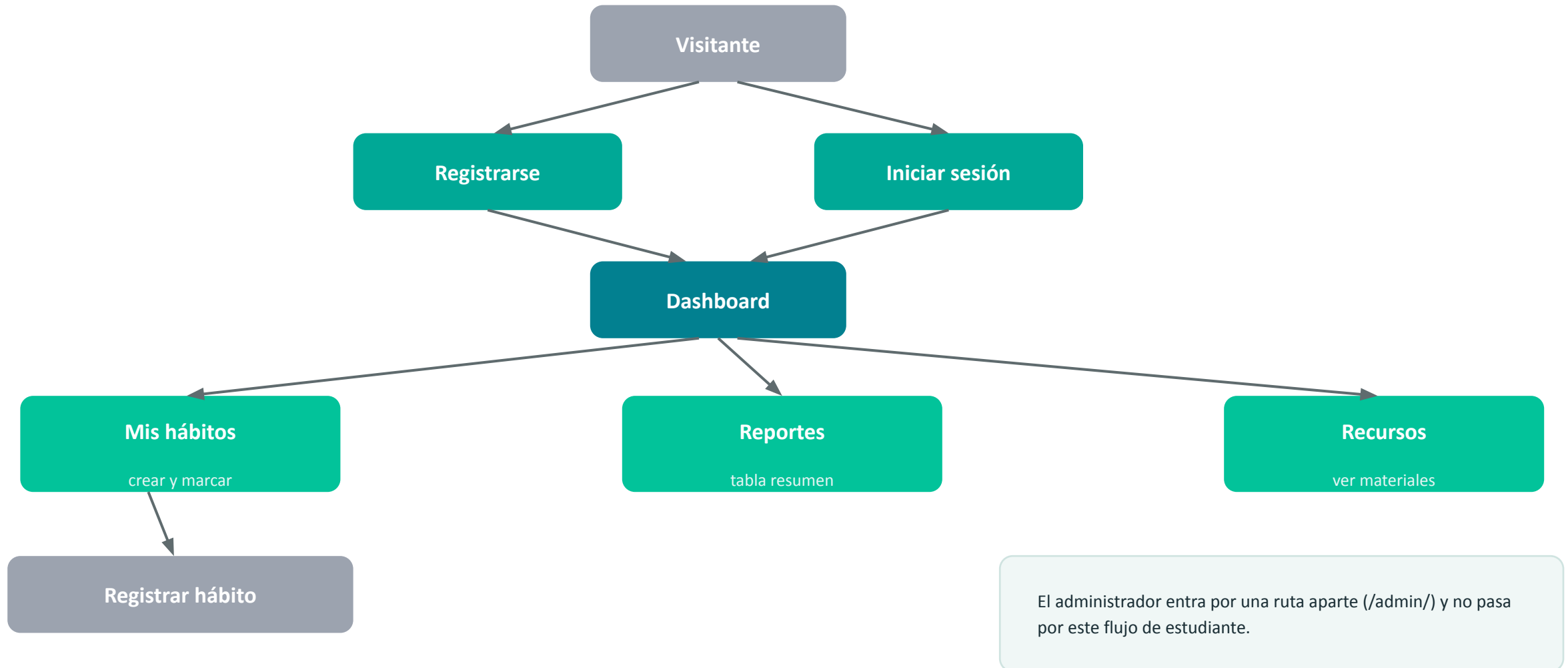
SQLite; el esquema queda versionado en migrations/

Apps modulares por dominio: accounts (usuarios) · habits (hábitos) · resources (recursos) — cada una con su propio `models` / `views` / `forms` / `urls`.

Modelo de datos — capa de persistencia



Flujo general del software



Principales desafíos y soluciones adoptadas

1

Extender el modelo de usuario de Django

Se creó un User personalizado (AbstractUser) con salt manual antes del hash — un punto donde es fácil duplicar trabajo que Django ya resuelve internamente.

2

El modelo de hábitos cambió de diseño a mitad de camino

Las migraciones muestran un diseño inicial más rico (tipos de hábito, estado de ánimo) simplificado luego a un hábito genérico — el alcance se replanteó sobre la marcha.

3

Curva de aprendizaje del framework

Coordinar formularios, validaciones de modelo (clean/full_clean) y migraciones de forma consistente entre tres apps distintas.

Errores de seguridad que no volveremos a cometer



SECRET_KEY hardcodedo

Quedó en texto plano dentro de settings.py en vez de leerse desde una variable de entorno.



DEBUG=True y ALLOWED_HOSTS=["*"]

Configuración de desarrollo que quedaría insegura si se despliega tal cual.



Backup de base de datos versionado

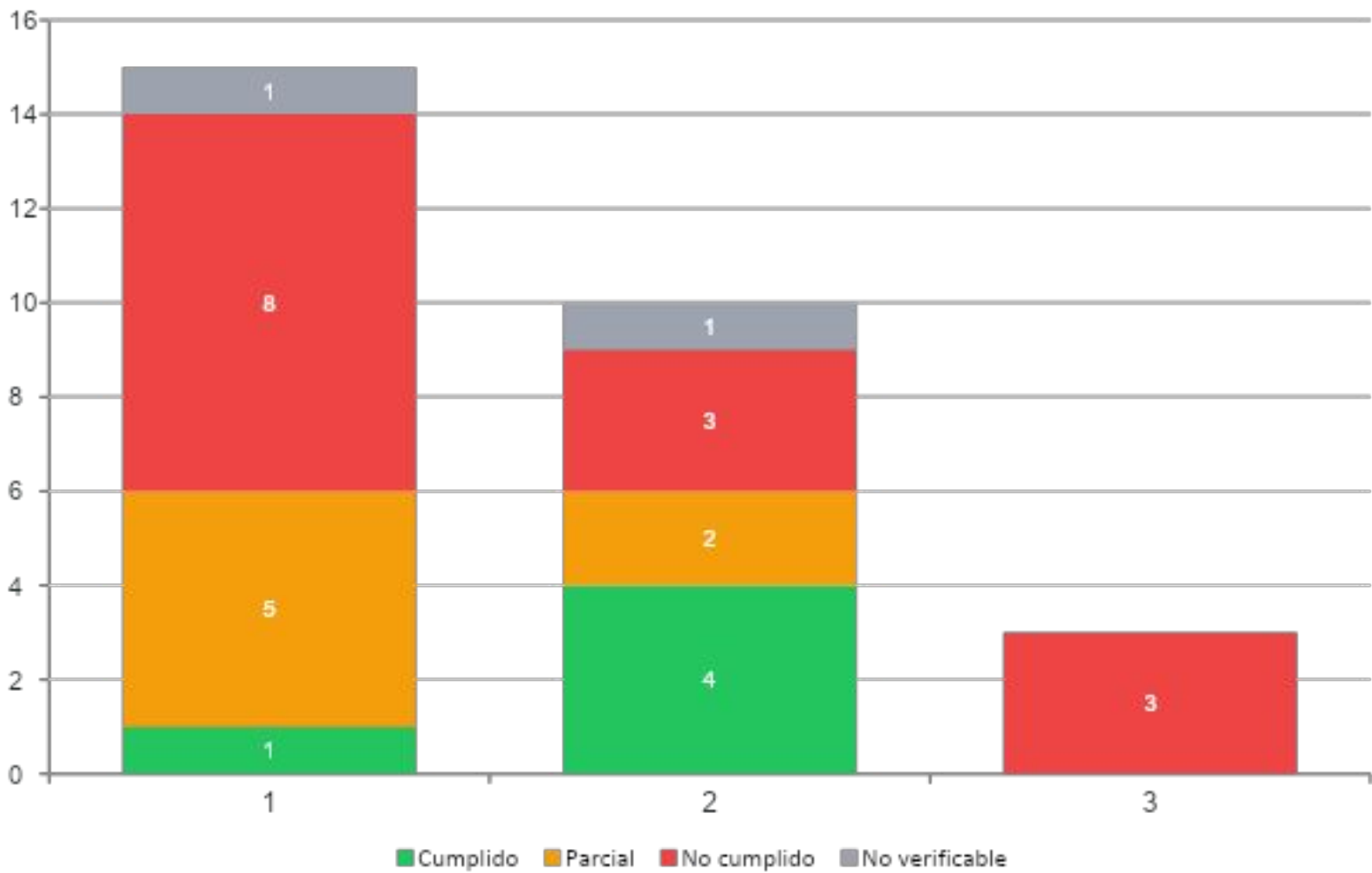
db.sqlite3.bak quedó en el repositorio, con datos reales de usuarios y contraseñas hasheadas.



Hash sin bcrypt

Se usa el PBKDF2 por defecto de Django, no bcrypt como se documentó en los requisitos.

Cumplimiento de requisitos (MoSCoW)



Lectura rápida

- Must have:** solo 1 de 15 cumplido al 100%.
- Should have:** 4 de 10 cumplidos.
- Could have:** 0 de 3 cumplidos.

El núcleo del producto (registro diario de sueño/agua/ánimo) fue reemplazado por un tracker genérico.

Distribución de la exposición — 4 integrantes

1

Daniel Felipe Agudelo Parra

Contexto y arquitectura general

Introducción del proyecto y las 3 capas (presentación / lógica / persistencia).

2

Omar Andrés Quiñones

Persistencia de datos

Modelo de datos, relaciones entre entidades y decisiones de diseño.

3

Jaime Javier Andrés Quiñones

Lógica de negocio y desafíos

Vistas, formularios, y los principales retos técnicos enfrentados.

4

Pedro Alejandro Castiblanco Castañeda

Estado del proyecto y cierre

Cumplimiento de requisitos, errores de seguridad y demo en vivo.

Gracias

¿Preguntas?